

The Basics of JavaScript

5

By the end of this section of the practical, the students should be able to:

- + Write JavaScript to generate HTML
- + Create simple scripts which include input and output statements, arithmetic, relational and equality operators.
- + Understand counter-controlled repetition and sentinel-controlled repetition.
- + Write functions and pass information between functions
- + Write programs that use JavaScript objects such as Math, String and Date.

Introduction

JavaScript, which was originally named LiveScript, was developed by Netscape. LiveScript became joint venture of Netscape and Sun Microsystems and its name was changed to JavaScript. Microsoft's JavaScript is named Jscript. The original goal of JavaScript was to provide programming capability at both the server and the client ends of a Web connection. Server-side JavaScript can serve as an alternative for some of what is done with server-side programming, in which computational capability resides on the server and is requested by the client. On the other hand, Client-side JavaScript cannot replace all server-side computing. It defines the collection of objects, methods, and properties that allow scripts to interact with HTML documents on the client. In this chapter, you will learn to write, test and debug Clientside JavaScript.

1.1 Hands-on Exercises

Javascript can enhance the dynamics and interactive features of your page by allowing you to perform calculations, check forms, write interactive games, add special effects, customize graphics selections, create security passwords and more. JavaScript is an Object Oriented Programming (OOP) language. An OOP language allows you to define your own objects and make your own variable types.

- **Objects and Properties**

Your web page document is just one of the many objects. Any table, form, button, image, or link on your page is also an object. Each object has certain properties (information about the object). For example, the background color of your document is written `document.bgcolor`. You would change the color of your page to red by writing the line: `document.bgcolor="red"`. The contents (or value) of a textbox named "password" in a form named "entryform" is `document.entryform.password.value`.

- **Methods**

Most objects have a certain collection of things that they can do. Different objects can do different things. A new document is opened with the method `document.open()`. You can write "Hello World" into a document by typing `document.write("Hello World")`. `open()` and `write()` are both methods of the object: `document`.

- **Events**

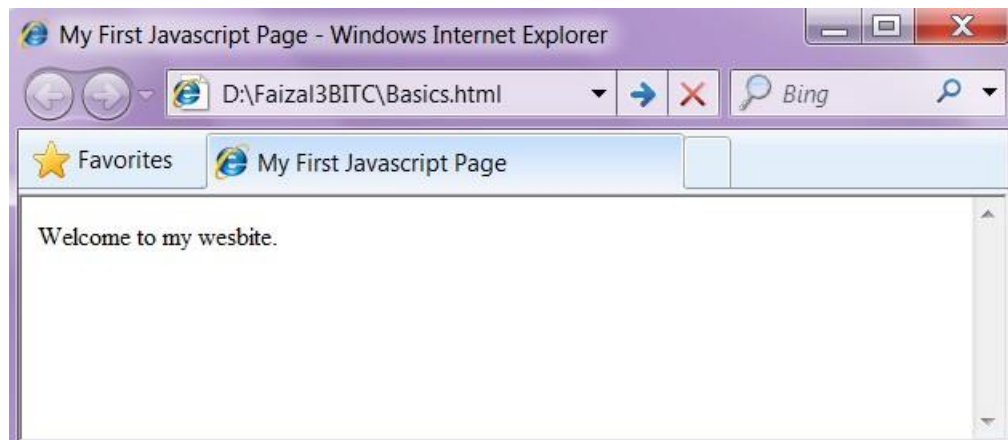
Events are how we trigger our functions to run. The easiest example is a button, whose definition includes the words `onClick="run_my_function()"`. The `onClick` event, as its name implies, will run the function when the user clicks on the button. Other events include `OnMouseOver`, `OnMouseOut`, `OnFocus`, `OnBlur`, `OnLoad`, and `OnUnload`.

1.1.1 Generating HTML & Displaying Variables

1. Open a text editor and type the following HTML document that contains JavaScript code. Save the file as „**Basic.html**“.

```
1 <html>
2 <head>
3 <title>My First Javascript Page</title>
4 </head>
5
6 <body>
7
8 <script type="text/javascript">
9 // Output Statement
10 document.write("Welcome to my wesbite.");
11 </script>
12
13 </body>
14 </html>
```

2. The HTML file executes the JavaScript statement to display the „**Welcome to my website.**“ text.



3. **Line 8, 11:** JavaScript code is included in an HTML document using the `<script></script>` tag.

4. **Line 10:** The word `'document.write'` is a standard JavaScript command for writing output to a page. "**document**" is a built-in object representing the HTML document where the JavaScript is embedded in. "**write(string)**" is a method provided by the "document" object. It writes the specified string into the HTML document represented by the "document" object.
5. **Line 9:** The two forward slashes at the end of comment line (`//`) are a JavaScript comment symbol. This prevents the JavaScript compiler from compiling the line.
6. Now, you will modify the above code. To do this, add a new output statement which formats the text "" with `<h1>` tag. Declare a JavaScript variable named **"title"** and assign the text **"Faizal's website."** as value to it, and then display the variable as shown below.

```
<html>
<head>
<title>My First Javascript Page</title>
</head>

<body>

<script type="text/javascript">
//declare variables
var title ="Faizal's Site";

// Output Statement
document.write("<h1>" + title + "</h1>");
document.write("<p>Welcome to my wesbite.</p>");
</script>

</body>
</html>
```

7. Your code should produce the following output on the browser.

Faizal's Site

Welcome to my wesbite.

1.1.2 Using an External JavaScript

1. The JavaScript you have created so far have only applied to the document itself. Internal scripts apply only to the document in which they were created. Now, you'll learn how to import an *external JavaScript* which contains the script you defined in respective document. The reason of creating external JavaScript is to enable user to run the same JavaScript on several pages, without having to write the same script on every page.
2. Open a new page on a text editor and copy all the JavaScript you have created in the '**Basic.html**' in Exercise 6.1.1. Save the external JavaScript file with an '**external.js**' file extension in the **same folder** with the html document. Please remember that the external script cannot contain the `<script>` tag.
3. To embed the external JavaScript in Basic.html, point to the '**external.js**' file in the '`src`' attribute of the `<script>` tag like below. Remember to place the script exactly where you normally would write the script.

```
<html>
<head>
<script type ="text/javascript" src="external.js"></script>
</head>
<body> </body>
</html>
```

4. Open the **Basic.html** on the browser. Your code should produce the output as same Exercise 6.1.1.

1.1.3 Built-in Alert, Confirm, Prompt Commands

As mentioned previously, JavaScript treats window as an object which has properties and method to access.

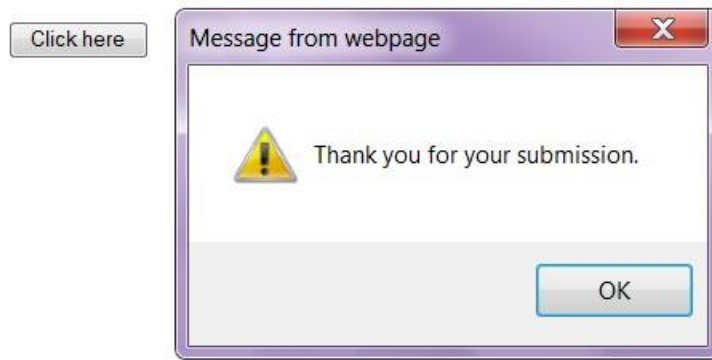
- Window (**PROPERTIES**): `document`, `bgcolor`, `fgcolor`, `default status`, `frame`, `parent`, `referrer`, `lastmodified`, `history`, `location`, `name` etc.
- Window (**METHODS**): `alert`, `prompt`, `confirm`, `write`, `writeln`, `close`, `open`, `setTimeout`, `reload`, `clearTimeout`, `blur`, `focus` etc.

The **alert**, **confirm** and **prompt** message boxes are methods of the interface window object. Because the window object is at the top of the object hierarchy, you do not have to use the full name (for example, `window.alert()`) of any of these dialog boxes. However, it is a good idea to use the full name, because it helps you remember which object the methods belong to.

1. To demonstrate this, type down this code below and save your file as **dialogbox.html**. The following code will add an **alert box** by using a HTML button and the **onClick** event.

```
<html>
<head><title>Dialog Boxes Demos</title>
<body>
<form name="myform">
<input type="button" value="Click here" onClick="window.alert('Thank
you for your submission.');">
</form>
</body>
</html>
```

2. Whatever you put inside the quotation marks in **windows.alert**, it will display it in the pop-up box. The button triggers an **alert method** when it's clicked. When a user triggers an **event**, your browser usually does something in response. Click a link, and it loads up the page in the link's href. Click into a form field and it receives the browser's focus. These are the default **event handlers** for each object in a HTML page. When we write our own event handlers, we write them directly into the HTML code so the browser knows which element we're addressing, and add in new commands on top of the existing default actions. Event Handlers aren't scripts in themselves, but often use variables, methods and functions from scripts to perform their action. You will learn more about event handlers in the next chapter.



3. Javascript **Alert box** are ideal for the following situations:
- If you want to be absolutely sure they see a message before doing anything on the website.
 - You would like to warn the user about something.
 - An error has occurred and you want to inform the user of the problem.
 - When asking the user for confirmation of some action.
4. Next, you are going to use a **Confirm box**. Confirmation are most often used to *confirm* an important actions that are taking place on a website. For example, they may be used to confirm an order submission or notify visitors that a link they clicked will take them to another website. Modify **dialogbox.html** by replacing the existing code with these blocks of snippets.

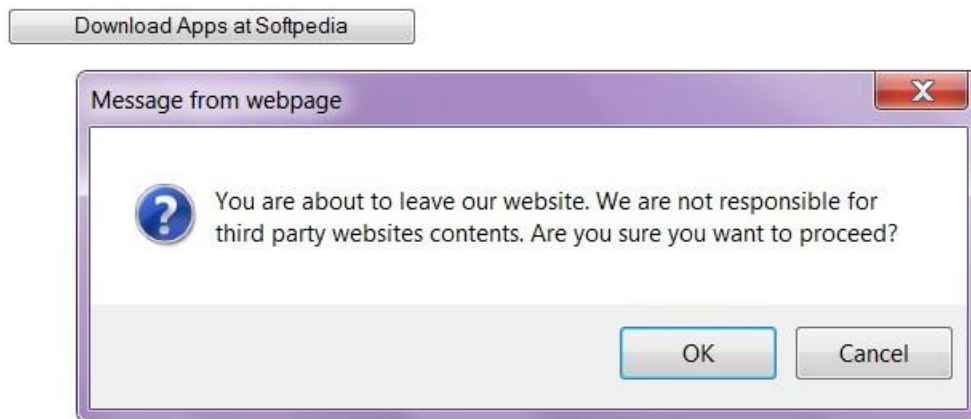
```
<html>
<head>
<title>Dialog Boxes and Event Handlers</title>
<script type="text/javascript">
function myConfirm() {
var answer = confirm("You are about to leave our site. We are not
responsible for a any third party websites contents. Are you sure you
want to proceed?");
    if(answer)
        { window.location="http://www.softpedia.com"; }
    else
        { document.write("Thanks for sticking around."); }
}
</script>
</head>
```

```

<body>
<form name="myform">
<input type="button" value="Download Apps at Softpedia"
onClick="myConfirm();" "> </form>
</body>
</html>

```

5. An interactive dialogue box will be displayed with the message-jump to another URL. If you click 'OK' you will be taken to that page because the **window.location** property is addressed to that place. If you click 'Cancel' then it will be cancelled.



6. Now, insert another button between the existing form tag.

```

<input type="button" value="Calculate the sum of 2 numbers"
onClick="myPrompt();" ">

```

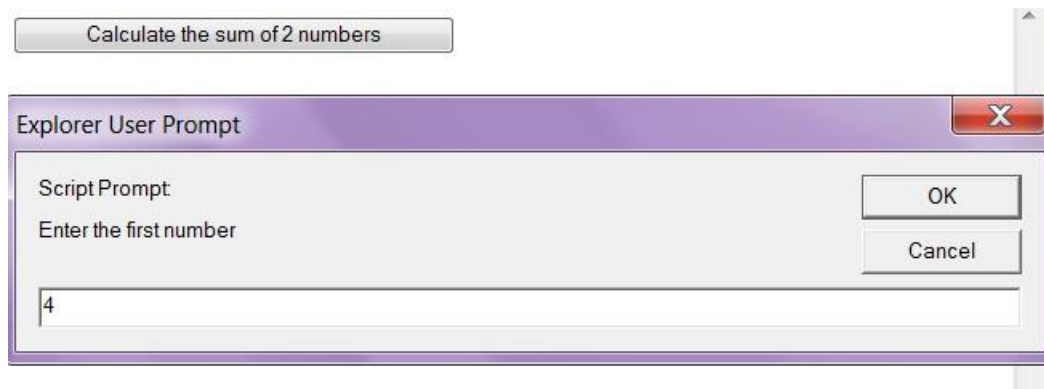
7. Finally, place another JavaScript function in the head section.

```

function myPrompt()
{
var num1, num2, result;
num1 = parseInt(window.prompt("Enter the first number","0"));
num2 = parseInt(window.prompt("Enter the second number","0"));
result = num1 + num2;
document.write('The sum of ' + num1 + ' and ' + num2 + ' is ' +
result);
}

```


8. The **Prompt box** is very useful when you want to pop-up a text box to get user input. Thus it enable you to interact with the user. The user needs to fill in the field and then click OK. The **parseInt()** function converts a string to an integer value.



9. Close your previous file and create a new HTML document. Insert this code and save it as **SwitchCase.html**

```
<html>
<head><title>More Examples</title></head>
<body>
<script type="text/javascript">
var number = prompt("Congratulations for being the winner of our
lucky draw contest. Enter a number between 1 to 3 to collect your
prize.", 0)
switch (number)
{
case (number = "1"):
    {document.write("You have won yourself an iPhone!");
    document.write("<br /><img
    src='http://farah.com.nu/pix/iphone.jpg'/> ");}
break;
case (number = "2"):
    {document.write("You have won yourself an iPod!");
    document.write("<br /><img
    src='http://farah.com.nu/pix/ipod.jpg'/>");}
break;
```

```

case (number = "3"):
    {document.write("You have won yourself an iPad!");
    document.write("<br /><img
    src='http://farah.com.nu/pix/ipad.jpg'/>");}
break;
default:
    alert("Sorry, wrong value.");
break;
}
</script>
</body>
</html>

```

10. So far, you have explored Javascript programming syntax such as variables, functions, operators, expressions, assignment, relational, logical arithmetic, operators, control structures such as `if.. else` and `switch..case` statements. Next, you will work with more arithmetic operations and iteration loops in your program.

1.1.4 Using Loop Statements

1. The *For* loop is used when you know in advance how many times the script should perform.

Type down the code below and save your file as **LoopFor.html**.

```

<html>
<head>
<title>Temperature Converter Table</title>
</head>

<body>
<table border=3>
<tr><td>CELSIUS</td><td>FAHRENHEIT</td></tr>
<script language="javascript">
for (celsius=-8; celsius<=47; celsius=celsius+1)
{

```

```

document.write("<tr><td>"+celsius+"</td><td>"
+ ((celsius*9/5)+32)+"</td></tr>");
}
</script>
</table>
</body>
</html>

```

2. The *While* loop is used when you want the loop to continue until a certain condition becomes true. Create a new HTML document named **LoopWhile.html** and write this code.

```

<html>
<head><title>Quick Quiz</title></head>
<body>
<script type="text/javascript">
var x=5*5;
x=prompt("What is 5 x 5?");
while (x != 25)
{
alert("Wrong answer");
x=prompt("What is 5 x 5?",0);
}
document.write("Correct, the answer is " + x)
</script>
</body>
</html>

```

3. You have declared variable *x* and initialized 25. Then, you set it equal to the prompt box. The prompt box asks the product of 5x5 and continues asking until true answer is provided. It also informs each wrong attempts or each loop



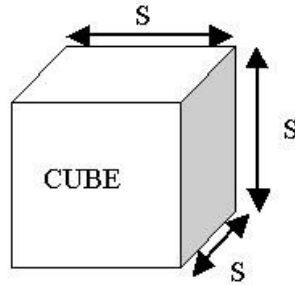
4 . Try modifying the current code using a `Do . . While` loop instead.

1.1.5 Working with JavaScripts Math Objects

Among the wide array of features offered by the Math object are several special values such as PI, natural logarithms, and common square roots, trigonometric methods, rounding methods, an absolute value method, and more. These can be used to make complex calculations in Javascript when needed.

Name	Usage
Abs()	Calculates the absolute value of a number.
Acos()	Calculates the arc cosine of a number—returns result in radians.
Asin()	Calculates the arc sine of a number—returns result in radians.
Atan()	Calculates the arc tangent of a number—returns result in radians.
Ceil()	Returns the next integer greater than or equal to a number.
Cos()	Calculates the cosine of a number.
Exp()	Calculates e to the power of a number.
floor()	Returns the next integer less than or equal to a number.
log()	Calculates the natural logarithm of a number.
max()	Returns the greater of two numbers—takes two arguments.
min()	Returns the least of two numbers—takes two arguments
pow()	Calculates the value of one number to the power of a second number—takes two arguments.
random()	Returns a random number between zero and one. Presently, this is only implemented in UNIX versions of Navigator 2.0.
round()	Rounds a number to the nearest integer.
sin()	Calculates the sine of a number.
sqrt()	Calculates the square root of a number.
tan()	Calculates the tangent of a number.

1. In this exercise, you are going to use the `pow()` method to calculate the volume of a cube based on the side's value.



2. Type down this code below and save your file as **Math.html**.

```
<html>
<head><title>Volume of Cube</title></head>
<body>

<script type = "text/javascript">
var s, vol;
s = parseInt(window.prompt("Enter the side of the volume","0"));
vol = Math.pow(s,3);
document.write("The volume of the cube is " + vol);
</script>

</body>
</html>
```